



Visão Computacional e Deep Learning

Ma. Cilene Renata Real
cilene.real@facens.br

Agenda

- Deep Learning
- Manipulação de Dados
- Operações
- Indexação e Fatiamento
- Pré-Processamento de Dados
- Álgebra Linear
- Cálculo
- Dataset de classificação de imagem
- Exercício

O que
aprendemos
na aula passada?



- Fundamentos de Visão Computacional
- Processamento de imagens com OpenCV

Aprendizagem Profunda ou Deep Learning, é uma sub-área da Aprendizagem de Máquina, que emprega algoritmos para processar dados e imitar o processamento feito pelo cérebro humano.

Deep Learning usa camadas de neurônios matemáticos para processar dados, compreender a fala humana e reconhecer objetos visualmente.

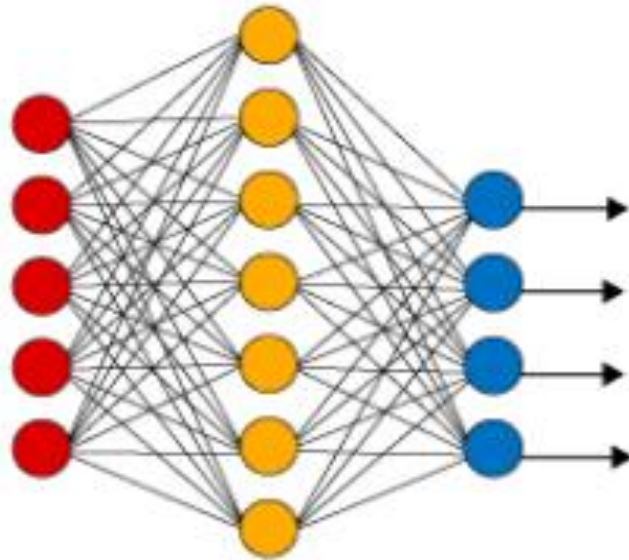
A informação é passada através de cada camada, com a saída da camada anterior fornecendo entrada para a próxima camada.

A primeira camada em uma rede é chamada de camada de entrada, enquanto a última é chamada de camada de saída.

Todas as camadas entre as duas são referidas como camadas ocultas. Cada camada é tipicamente um algoritmo simples e uniforme contendo um tipo de função de ativação.

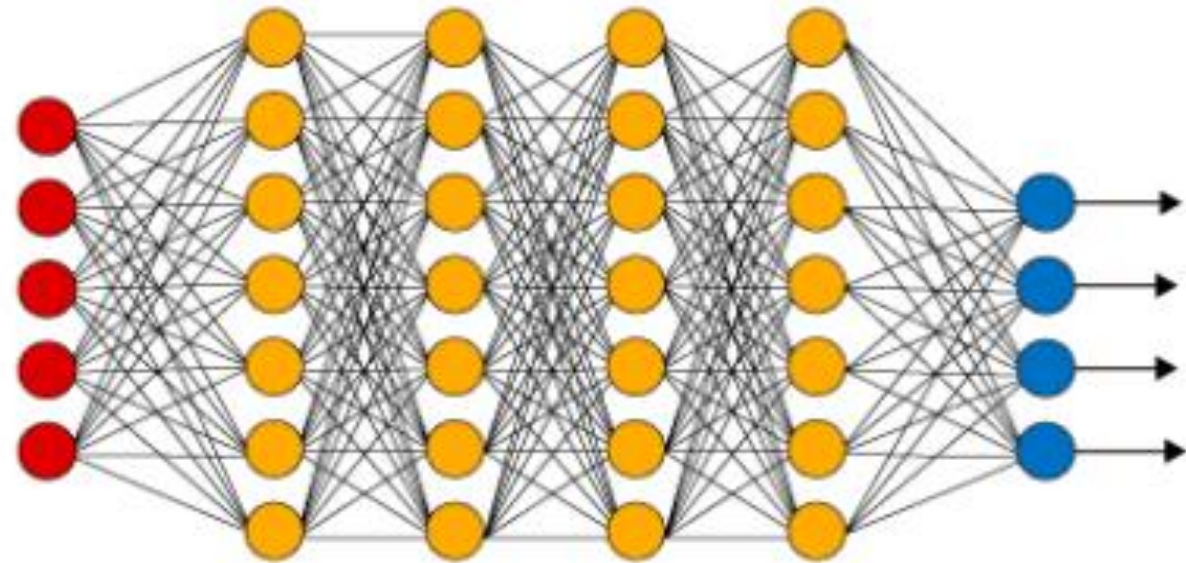
Deep Learning

Simple Neural Network



● Input Layer

Deep Learning Neural Network



● Hidden Layer

● Output Layer

A aprendizagem profunda é responsável por avanços recentes em visão computacional, reconhecimento de fala, processamento de linguagem natural e reconhecimento de áudio.

O aprendizado profundo é baseado no conceito de redes neurais artificiais, ou sistemas computacionais que imitam a maneira como o cérebro humano funciona.

A extração de recursos é outro aspecto da Aprendizagem Profunda. A extração de recursos usa um algoritmo para construir automaticamente “recursos” significativos dos dados para fins de treinamento, aprendizado e compreensão.

Para iniciarmos o nosso aprendizado de Deep Learning, precisaremos desenvolver algumas habilidades básicas.

Todo aprendizado de máquina está relacionado com a extração de informações dos dados. Portanto, começaremos aprendendo as habilidades práticas para armazenar, manipular e pré-processar dados.

Além disso, o aprendizado de máquina normalmente requer trabalhar com grandes conjuntos de dados, que podemos considerar como tabelas, onde as linhas correspondem a exemplos e as colunas correspondem aos atributos.

A álgebra linear nos dá um poderoso conjunto de técnicas para trabalhar com dados tabulares. Nos concentraremos no básico das operações matriciais e sua implementação.

Vamos abrir o Colab, por gentileza e criar um novo notebook.

Para fazer qualquer coisa, precisamos de alguma forma de armazenar e manipular dados.

Geralmente, há duas coisas importantes que precisamos fazer com os dados:

- adquirir eles;
- processá-los.

Para começar, apresentamos o array n -dimensional, também chamado de tensor.

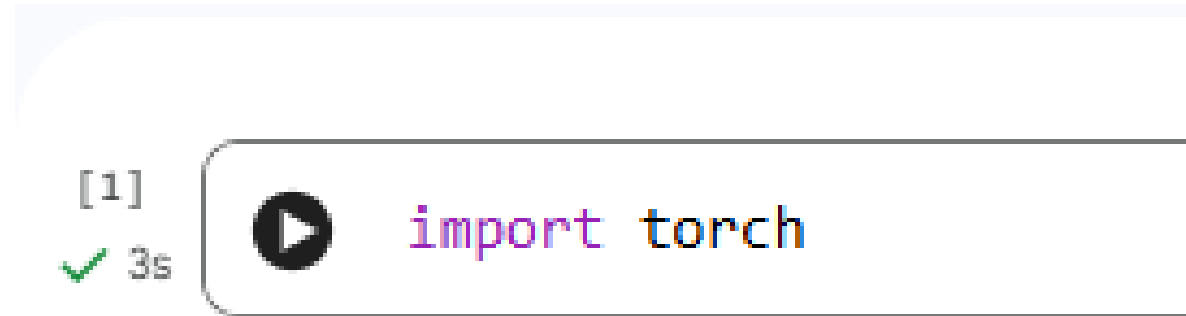
Não importa qual estrutura você usa, sua classe de tensor (ndarray em MXNet, Tensor em PyTorch e TensorFlow) é semelhante ao ndarray do NumPy.

Nessa aula utilizaremos o PyTorch, por ele permitir aprendizado profundo.

Para começar, importamos torch.

Note que apesar de ser chamado PyTorch, devemos importar torch ao invés de pytorch.

Primeiro comando no colab:



Um tensor representa uma matriz (possivelmente multidimensional) de valores numéricos.

Com uma dimensão, um tensor corresponde (em matemática) a um vetor. Com duas dimensões, um tensor corresponde a uma matriz.

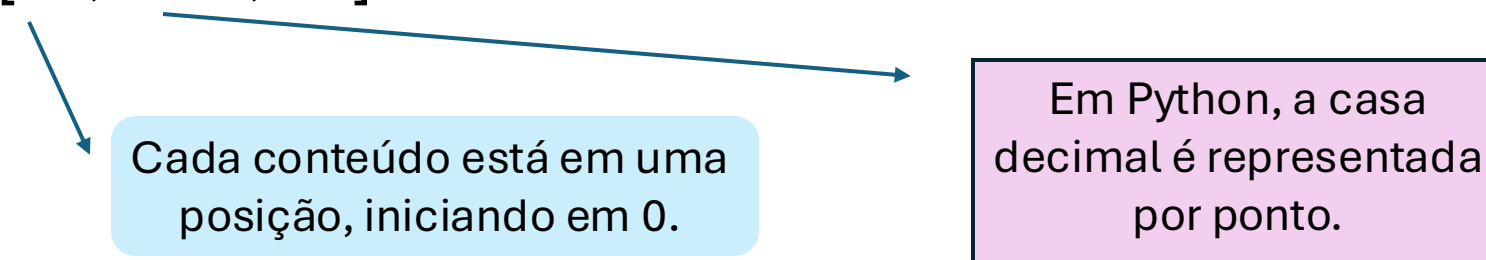
Vetor: uma lista de conteúdos com uma dimensão.

Por exemplo, imagine que queremos representar um aluno por três características:

- Idade: 20 anos
- Altura: 1,75 m
- Peso: 70 kg

Podemos escrever essas informações em um vetor:

[20, 1.75, 70]



Cada conteúdo está em uma posição, iniciando em 0.

Em Python, a casa decimal é representada por ponto.

Uma **matriz** é um conjunto de vetores organizados em linhas e colunas.

Exemplo:

Idade	Altura	Peso
20	1,75	70
22	1,80	80
19	1,68	60

Matriz:

$$\begin{bmatrix} 20, 1.75, 70 \\ 22, 1.80, 80 \\ 19, 1.68, 60 \end{bmatrix}$$

Nós seres humanos conseguimos visualizar uma matriz de até 3 dimensões.

Porém, o armazenamento e processamento pode ocorrer considerando n dimensões.

Para começar, podemos usar **arange** para criar um vetor linha x contendo os primeiros 12 inteiros começando com 0:

```
x = torch.arange(12)
print(x)
```

Posso criar um vetor manualmente:

```
y= [8, 5, 12, 26, 59, 587, 10, 22]
print(y)
```

Para saber o tamanho o primeiro vetor:

```
x.shape
```

Para mudar o shape de um tensor sem alterar o número de elementos ou seus valores, podemos invocar a função ***reshape***.

Por exemplo, podemos transformar nosso tensor, `x`, de um vetor linha para uma matriz com forma (3, 4).

Este novo tensor contém exatamente os mesmos valores, mas os vê como uma matriz organizada em 3 linhas e 4 colunas.

```
X = x.reshape(3, 4)
print(X)
```


Podemos criar uma matriz apenas com zeros:

```
torch.zeros(12, 3)
```

Ou uma matriz apenas com números 1:

```
torch.ones(2, 8)
```

E uma matriz com números aleatórios:

```
torch.randn(3, 4)
```

Podemos também especificar os valores exatos para cada elemento no tensor desejado:

```
torch.tensor([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
```

Os **operadores aritméticos** padrão comuns são: **+**, **-**, *****, **/** e ******.

```
x = torch.tensor([1.0, 2, 4, 8])  
y = torch.tensor([2, 2, 2, 2])  
x + y, x - y, x * y, x / y, x ** y
```

Também podemos concatenar vários tensores juntos, empilhando-os ponta a ponta para formar um tensor maior.

Só precisamos fornecer uma lista de tensores e informar ao sistema ao longo de qual eixo concatenar.

```
▶ X = torch.arange(12, dtype=torch.float32).reshape((3,4))  
Y = torch.tensor([[2.0, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])  
torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)
```

O exemplo mostra o que acontece quando concatenamos duas matrizes ao longo das linhas (eixo 0, o primeiro elemento da forma) vs. colunas (eixo 1, o segundo elemento da forma).

Podemos ver que o comprimento do eixo 0 do primeiro tensor de saída (6) é a soma dos comprimentos do eixo 0 dos dois tensores de entrada (3+3); enquanto o comprimento do eixo 1 do segundo tensor de saída (8) é a soma dos comprimentos do eixo 1 dos dois tensores de entrada (4+4).

Operadores relacionais são usados para comparar valores.

O resultado de uma comparação é sempre um valor booleano: **True** (verdadeiro) ou **False** (falso).

`==` Igual

`!=` Diferente

`>=` Maior igual

`<=` Menor igual

`>` Maior

`<` Menor



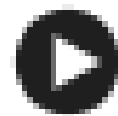
`X == Y`

Assim como em qualquer outro array Python, os elementos em um tensor podem ser acessados por índice.

O primeiro elemento tem índice 0.

Podemos acessar os elementos de acordo com sua posição relativa ao final da lista usando índices negativos.

Assim, `[-1]` seleciona o último elemento e `[1: 3]` seleciona o segundo e o terceiro elementos da seguinte forma:



`X[-1], X[1:3]`

Se quisermos para atribuir a vários elementos o mesmo valor, simplesmente indexamos todos eles e, em seguida, atribuímos o valor a eles.

Por exemplo, `[0: 2,:]` acessa a primeira e a segunda linhas, onde: leva todos os elementos ao longo do eixo 1 (coluna).

Enquanto discutimos a indexação de matrizes, isso obviamente também funciona para vetores e para tensores de mais de 2 dimensões.

```
X[0:2, :] = 12
```

X

Pré-Processamento de Dados

Para aplicar o Deep Learning na solução de problemas do mundo real, frequentemente começamos com o pré-processamento de dados brutos, em vez desses dados serem configurados no formato tensor.

Entre as ferramentas analíticas de dados populares em Python, o pacote ***pandas*** é comumente usado

Criamos um conjunto de dados artificiais que são armazenados em um arquivo csv (valores separados por vírgula).

```
import os

os.makedirs(os.path.join('.', 'data'), exist_ok=True)
data_file = os.path.join('.', 'data', 'teste.csv')
with open(data_file, 'w') as f:
    f.write('Num_Linha,Cod,Descricao,Preco\n') # Column names
    f.write('1,10,Laranja,12.75\n') # Each row represents a data example
    f.write('2,20,Banana,NA\n')
    f.write('3,30,Pera,17.81\n')
    f.write('4,NA,Uva,14.00\n')
```


Para carregar o conjunto de dados bruto do arquivo csv criado, importamos o pacote ***pandas*** e chamamos a função ***read_csv***.

```
# Se o pandas ainda não estiver instalado descomente a linha abaixo:  
# !pip install pandas  
import pandas as pd  
  
data = pd.read_csv(data_file)  
print(data)
```

Um escalar é representado por um tensor com apenas um elemento.

No próximo trecho de código, instanciamos dois escalares e realizar algumas operações aritméticas familiares com eles, a saber, adição, multiplicação, divisão e exponenciação.

```
import torch

x = torch.tensor([3.0])
y = torch.tensor([2.0])

x + y, x * y, x / y, x**y
```

Você pode pensar em um vetor simplesmente como uma **lista de valores escalares**.

Chamamos esses valores de elementos (entradas ou componentes) do vetor. Quando nossos vetores representam exemplos de nosso conjunto de dados, seus valores têm algum significado no mundo real.

Por exemplo, se estivéssemos treinando um modelo para prever o risco de inadimplência de um empréstimo, podemos associar cada candidato a um vetor cujos componentes correspondem à sua receita, tempo de emprego, número de inadimplências anteriores e outros fatores.

Se estivéssemos estudando o risco de ataques cardíacos que os pacientes de hospitais potencialmente enfrentam, podemos representar cada paciente por um vetor cujos componentes capturam seus sinais vitais mais recentes, níveis de colesterol, minutos de exercício por dia, entre outros.

Podemos criar uma matriz especificando uma forma com dois componentes e ao chamar qualquer uma de nossas funções favoritas para instanciar um tensor.

```
A = torch.arange(20).reshape(5, 4)  
A
```

Às vezes, queremos inverter os eixos.

Quando trocamos as linhas e colunas de uma matriz, o resultado é chamado de **transposição da matriz**.



$A.T$

Como um tipo especial de matriz quadrada, a **matriz simétrica A** é igual à sua transposta: $\mathbf{A} = \mathbf{A}^T$.

```
B = torch.tensor([[1, 2, 3], [2, 0, 4], [3, 4, 5]])  
B
```

Agora comparamos B com sua transposta:



```
B == B.T
```

Assim como vetores generalizam escalares e matrizes generalizam vetores, podemos construir estruturas de dados com ainda mais eixos.

Tensores (“tensores” nesta subseção referem-se a objetos algébricos), nos dão uma maneira genérica de descrever matrizes n -dimensionais com um número arbitrário de eixos.

Os tensores se tornarão mais importantes quando começarmos a trabalhar com imagens, que chegam como matrizes n -dimensionais com 3 eixos correspondentes à altura, largura e um eixo de canal para empilhar os canais de cores (vermelho, verde e azul).

Você deve ter notado da definição de uma operação elemento a elemento que qualquer operação unária elementar não altera a forma de seu operando.

Similarmente, dados quaisquer dois tensores com a mesma forma, o resultado de qualquer operação binária elementar será um tensor da mesma forma.

Por exemplo, adicionar duas matrizes da mesma forma realiza a adição elemento a elemento sobre essas duas matrizes.

```
▶ A = torch.arange(20, dtype=torch.float32).reshape(5, 4)
  B = A.clone() # Cria uma cópia de `A` em `B` alocando nova memória
  A, A + B
```

Especificamente, a multiplicação elemento a elemento de duas matrizes é chamada de **produto**.



Multiplicar ou adicionar um tensor por um escalar também não muda a forma do tensor, onde cada elemento do tensor operando será adicionado ou multiplicado pelo escalar.

```
a = 2
X = torch.arange(24).reshape(2, 3, 4)
a + X, (a * X).shape
```


Uma operação útil que podemos realizar com tensores arbitrários é para calcule a soma de seus elementos.

No código, podemos apenas chamar a função para calcular a soma.

```
▶ x = torch.arange(4, dtype=torch.float32)  
x, x.sum()
```

Por padrão, invocar a função para calcular a soma reduz um tensor ao longo de todos os seus eixos a um escalar.

Também podemos especificar os eixos ao longo dos quais o tensor é reduzido por meio da soma. Para reduzir a dimensão da linha (eixo 0) somando os elementos de todas as linhas, especificamos `axis = 0` ao invocar a função.

```
▶ A_sum_axis0 = A.sum(axis=0)  
A_sum_axis0, A_sum_axis0.shape
```

Reduzindo uma matriz ao longo de ambas as linhas e colunas por meio da soma é equivalente a somar todos os elementos da matriz.

▶ `A.sum(axis=[0, 1])` # O mesmo que `A.sum()`

Uma quantidade relacionada é a média.
Calculamos a média dividindo a soma pelo número total de elementos.

▶ `A.mean(), A.sum() / A.numel()`

Da mesma forma, a função de cálculo da média também pode reduzir um tensor ao longo dos eixos especificados.

▶ `A.mean(axis=0), A.sum(axis=0) / A.shape[0]`

Contudo, às vezes pode ser útil manter o número de eixos inalterado ao invocar a função para calcular a soma ou média.

▶ `sum_A = A.sum(axis=1, keepdims=True)`
`sum_A`

Por exemplo, uma vez que `sum_A` ainda mantém seus dois eixos após somar cada linha, podemos dividir `A` por `sum_A`.

▶ `A / sum_A`

Se quisermos calcular a soma cumulativa dos elementos de `A` ao longo de algum eixo, podemos chamar a função ***cumsum***.

▶ `A.cumsum(axis=0)`

Uma das operações mais fundamentais é o produto escalar.

Dado dois vetores, seu produto escalar é uma soma sobre os produtos dos elementos na mesma posição:

```
▶ y = torch.ones(4, dtype = torch.float32)  
x, y, torch.dot(x, y)
```

Podemos expressar o produto escalar de dois vetores de forma equivalente, realizando uma multiplicação elemento a elemento e, em seguida, uma soma:

```
▶ torch.sum(x * y)
```

Alguns dos operadores mais úteis em álgebra linear são normas.

Informalmente, a norma de um vetor nos diz o quão grande é um vetor.

A noção de tamanho em consideração aqui preocupa-se não em dimensionalidade, mas sim a magnitude dos componentes.

Na álgebra linear, uma norma vetorial é uma **função f** que mapeia um vetor para um escalar, satisfazendo um punhado de propriedades.

Em Deep Learning, trabalhamos com mais frequência com a norma **L_2** ao quadrado.

A L_2 norma de $\mathbf{x}$ é a raiz quadrada da soma dos quadrados dos elementos do vetor:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2},$$

onde o subscrito 2 é frequentemente omitido nas normas L_2 , ou seja, $\|\mathbf{x}\|$ é equivalente a $\|\mathbf{x}\|_2$. Em código, podemos calcular a norma L_2 de um vetor da seguinte maneira.



```
u = torch.tensor([3.0, -4.0])  
torch.norm(u)
```


Encontrar a área de um polígono permaneceu um mistério até pelo menos 2.500 anos atrás, quando os gregos antigos dividiam um polígono em triângulos e somavam suas áreas.

Em Deep Learning, nós treinamos modelos, atualizando-os sucessivamente para que fiquem cada vez melhores à medida que veem mais e mais dados. Normalmente, melhorar significa minimizar uma função de perda, uma pontuação que responde à pergunta “quão ruim é o nosso modelo?”

Esta pergunta é mais sutil do que parece.

Em última análise, o que realmente nos preocupa está produzindo um modelo com bom desempenho em dados que nunca vimos antes. Mas só podemos ajustar o modelo aos dados que podemos realmente ver.

Assim, podemos decompor a tarefa de ajustar os modelos em duas preocupações principais:

- i) **otimização**: processo de adequação de nossos modelos aos dados observados;
- ii) **generalização**: os princípios matemáticos e a sabedoria dos profissionais que guia sobre como produzir modelos cuja validade se estende além do conjunto exato de exemplos de dados usados para treiná-los.

O **cálculo de derivadas**, uma etapa crucial em quase todos os algoritmos de otimização de Deep Learning.

No Deep Learning, normalmente escolhemos funções de perda que são diferenciáveis em relação aos parâmetros do nosso modelo.

Simplificando, isso significa que para cada parâmetro, podemos determinar a rapidez com que a perda aumentaria ou diminuiria, deveríamos aumentar ou diminuir esse parâmetro por uma quantidade infinitesimalmente pequena.

Calculando a derivada:

```
import sympy as sp

# Define a variável simbólica
x = sp.symbols('x')

# Define a função
f = 3*x**2 - 4*x

# Calcula a derivada
df = sp.diff(f, x)

# Exibe a função e sua derivada
print("f(x) =", f)
print("f'(x) =", df)
```

```
import sympy as sp # Garante que sympy esteja disponível nesta célula

def numerical_lim(func, x_val, h):
    return (func(x_val + h) - func(x_val)) / h

# Converte a função simbólica 'f' (definida na célula anterior) em uma função numérica
# 'x' é a variável simbólica também definida na célula anterior.
f_numerical = sp.lambdify(x, f, 'numpy')

h = 0.1
for i in range(5):
    # Passa a versão numérica da função para numerical_lim
    print(f'h={h:.5f}, numerical limit={numerical_lim(f_numerical, 1, h):.5f}')
    h *= 0.1
```

O código demonstra numericamente como a derivada é definida como um limite e mostra que a inclinação da reta tangente em $x=1$ tende a 2.

O aprendizado de máquina envolve compreensão.

Podemos prever a **probabilidade** de um paciente sofrer um ataque cardíaco no próximo ano, considerando seu histórico clínico.

Na detecção de anomalias, podemos avaliar com que probabilidade seria um conjunto de leituras do motor do jato de um avião, se ele estivesse operando normalmente.

Na aprendizagem por reforço, queremos que um agente esteja já de forma inteligente em um ambiente.

Isso significa que precisamos pensar sobre a probabilidade de obter uma alta recompensa em cada uma das ações disponíveis.

E quando construímos sistemas de recomendação, também precisamos pensar sobre a probabilidade.

Cálculo

Considerando o primeiro caso: distinguir gatos com base em fotografias.

Isso pode parecer simples, mas na verdade é um desafio formidável.

Para começar, a dificuldade do problema pode depender da resolução da imagem:



Podemos visualizar como essas probabilidades convergem ao longo do tempo para a probabilidade verdadeira.

```
import matplotlib.pyplot as plt
from torch.distributions import multinomial
import torch

# Probabilidades de um dado honesto
fair_probs = torch.ones(6) / 6

# Simula 500 grupos de 10 lançamentos
counts = multinomial.Multinomial(10, fair_probs).sample((500,))
cum_counts = counts.cumsum(dim=0)
estimates = cum_counts / cum_counts.sum(dim=1, keepdims=True)

# Define o tamanho da figura
plt.figure(figsize=(6, 4.5))

# Plota a probabilidade estimada para cada face
for i in range(6):
    plt.plot(estimates[:, i].numpy(),
            label=f'P(die={i+1})')

# Linha horizontal indicando a probabilidade teórica (1/6)
plt.axhline(y=1/6, color='black', linestyle='dashed')

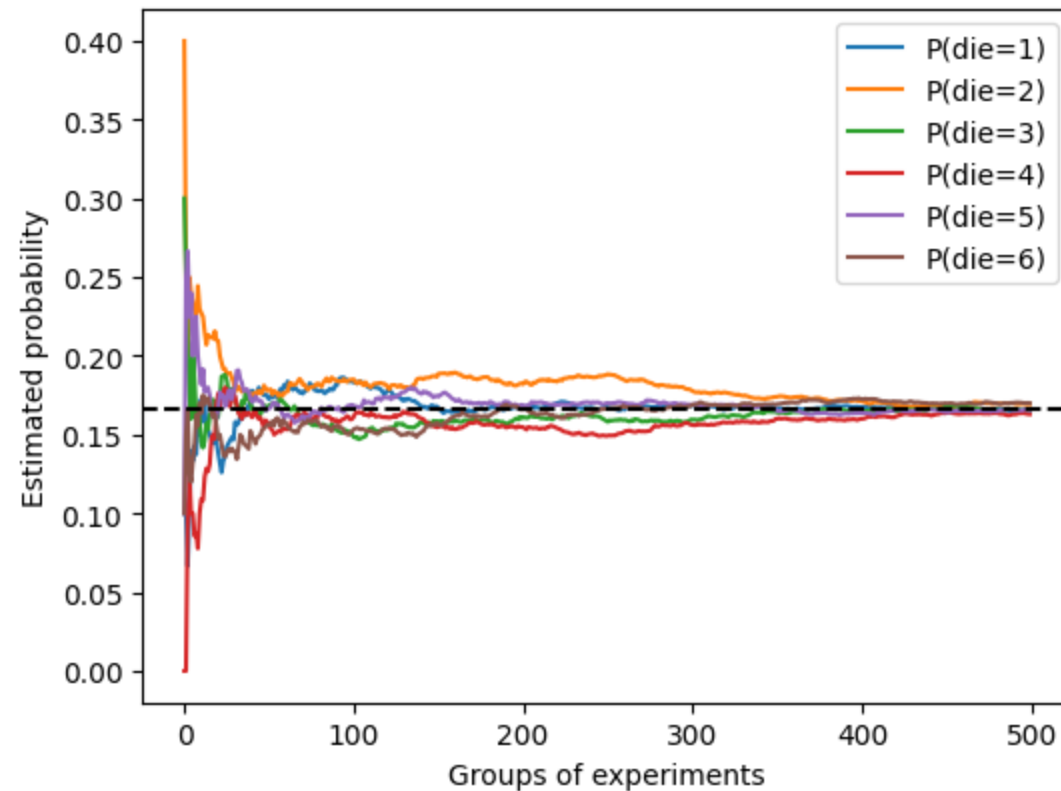
# Rótulos dos eixos
plt.xlabel('Groups of experiments')
plt.ylabel('Estimated probability')

# Legenda
plt.legend()

# Exibe o gráfico
plt.show()
```


Cálculo

Cada curva sólida corresponde a um dos seis valores do dado e dá nossa probabilidade estimada de que o dado aumente esse valor conforme avaliado após cada grupo de experimentos.



Dataset de classificação de imagem

Um dos conjuntos de dados amplamente usados para classificação de imagens é o conjunto de dados MNIST [LeCun et al., 1998] .

Embora tenha tido uma boa execução como um conjunto de dados de referência, mesmo os modelos simples pelos padrões atuais alcançaram uma precisão de classificação acima de 95%, tornando-o inadequado para distinguir entre modelos mais fortes e mais fracos.

```
%matplotlib inline  
  
import torch  
import torchvision  
from torch.utils import data  
from torchvision import transforms  
import matplotlib.pyplot as plt
```

Podemos baixar e ler o conjunto de dados Fashion-MNIST na memória por meio das funções integradas na estrutura.

```
trans = transforms.ToTensor()  
mnist_train = torchvision.datasets.FashionMNIST(  
    root="../../data", train=True, transform=trans, download=True)  
mnist_test = torchvision.datasets.FashionMNIST(  
    root="../../data", train=False, transform=trans, download=True)
```

Dataset de classificação de imagem

O **Fashion-MNIST** consiste em imagens de 10 categorias, cada uma representada por 6.000 imagens no conjunto de dados de treinamento e por 1.000 no conjunto de dados de teste.

Um dataset de teste é usado para avaliar o desempenho do modelo e não para treinamento.

Consequentemente, o conjunto de treinamento e o conjunto de teste contém 60.000 e 10.000 imagens, respectivamente.

```
len(mnist_train), len(mnist_test)
```

Dataset de classificação de imagem

A altura e a largura de cada imagem de entrada são 28 pixels.

Observe que o conjunto de dados consiste em imagens em tons de cinza, cujo número de canais é 1

```
mnist_train[0][0].shape
```

Dataset de classificação de imagem

As imagens no Fashion-MNIST estão associadas às seguintes categorias: camiseta, calças, pulôver, vestido, casaco, sandália, camisa, tênis, bolsa e bota.

A função a seguir converte entre índices de rótulos numéricos e seus nomes em texto.

```
def get_fashion_mnist_labels(labels):  
    """Return text labels for the Fashion-MNIST dataset."""  
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',  
                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']  
    return [text_labels[int(i)] for i in labels]
```

Dataset de classificação de imagem

Podemos criar uma função para visualizar esses exemplos:

```
import matplotlib.pyplot as plt
import torch
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plota uma lista de imagens."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    axes = axes.flatten()

    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Imagem em tensor
            ax.imshow(img.numpy())
        else:
            # Imagem PIL
            ax.imshow(img)

        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)

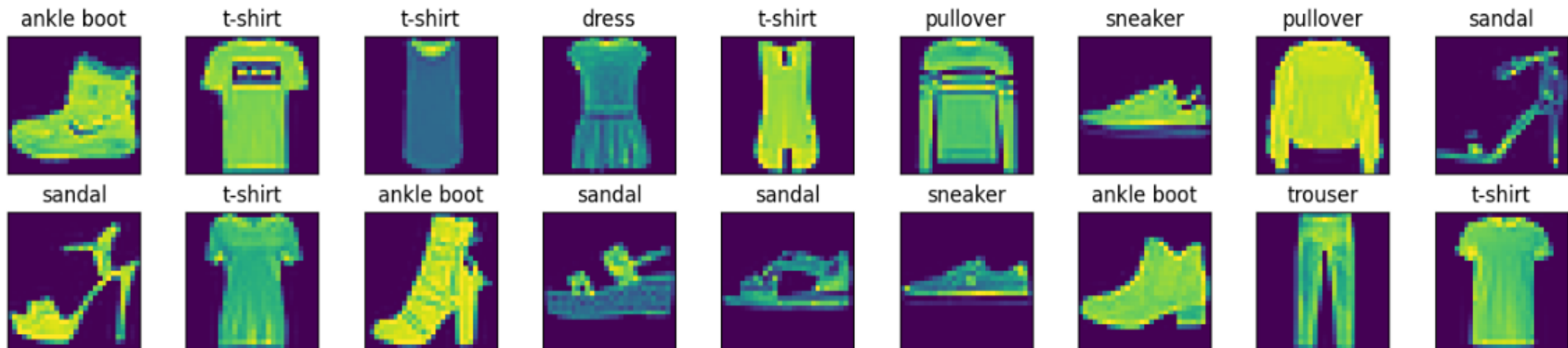
    if titles:
        ax.set_title(titles[i])

    plt.tight_layout()
    return axes
```

Dataset de classificação de imagem

As imagens e seus rótulos estão correspondentes (no texto) para os primeiros exemplos no dataset* de treinamento:

```
X, y = next(iter(data.DataLoader(mnist_train, batch_size=18)))  
show_images(X.reshape(18, 28, 28), 2, 9, titles=get_fashion_mnist_labels(y));
```



Dataset de classificação de imagem

Parte da imagem 1, considerando o fatiamento da matriz:

```
X, y = next(iter(data.DataLoader(mnist_train, batch_size=18)))  
  
# Primeira imagem, metade superior esquerda  
img_parcial = X[0, 0, :19, :19]  
  
show_images(img_parcial.reshape(1, 19, 19), 1, 1,  
            titles=[get_fashion_mnist_labels(y)[0]]);
```

Exercício

1-) Para o dataset utilizado no exemplo, exiba as últimas imagens.

2-) Exiba mais dois conjuntos de imagens intermediários.

3-) Exiba imagens aleatórias?

4-) Busque por 3 datasets diferentes e realize exposições deles.

Exs: CIFAR-10, KMNIST (caracteres japoneses), EMNIST (letras e dígitos), CelebA (rostos).